AWS S3 and IAM: Access-Control Lab

Younes Chemmaoui
Prepared: 7 November 2026
Evidence set: captures dated 29 October - 7 November 2026

1. Objective

I investigated the difference between IAM identity-based permissions and S3 resource-based permissions. The practical questions were whether devuser could create and write to a bucket, what changed after assuming a role, and which role the bucket3 challenge policy named.

2. Environment and scope

This was a preconfigured AWS training exercise. The environment supplied the IAM user devuser, DeveloperGroup, policies, roles, and the three lab buckets. The report does not claim that I designed or provisioned those resources. S3 captures show US East (N. Virginia). Account and generated resource identifiers are intentionally withheld.

The source folder contains captures from more than one training session. I use generic bucket1, bucket2 and bucket3 labels and keep the distinction between an observed Console outcome and an explanation inferred from a policy. The later captures extend the older written report.

3. Method

I attempted Console operations as devuser, inspected the group policy, switched to BucketsAccessRole, and compared object-access results. I reviewed the bucket2 resource policy and, in the later capture set, the bucket3 policy naming OtherBucketAccessRole. No new AWS operation was run to prepare this report.

Principal and operation	Result	Basis
devuser: create bucket	Allowed	Notes and source creation capture
devuser: upload file	Denied	Failed upload and group-policy capture
devuser: read bucket1 object	Denied	Source XML authorization error
BucketsAccessRole: bucket1 download	Allowed	Completed Image2.jpg download
BucketsAccessRole: bucket2 upload	Allowed	Later upload-success banner
OtherBucketAccessRole: bucket3 upload	One file succeeded	Later upload summary and matching policy principal

Table 1. Recorded outcomes. These are Console observations, not an independently replayed API test suite.

4. Observations and results

4.1 IAM user: bucket creation and upload are separate permissions

The group-policy capture includes s3:CreateBucket and listing permissions but no s3:PutObject. The notes record successful bucket creation, followed by a denied local-file upload. The denied upload capture shows a PNG file, so I do not describe it as a completed JSON-policy-file upload simply because the supplied exercise requested one.



Figure 1. Captured DeveloperGroupPolicy. Bucket creation and listing actions are visible; no object-write action is listed.

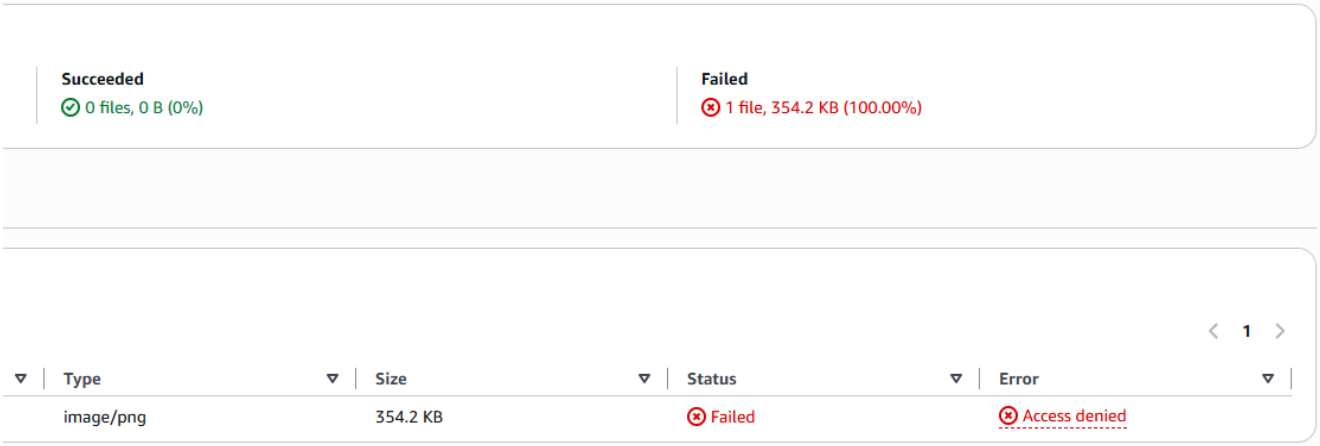


Figure 2. Cropped upload result: zero files succeeded, one failed, and the file row reports Access denied. The notes associate this attempt with devuser. The crop removes unrelated account and bucket context.

The observed denial is consistent with the missing object-write permission. It does not, by itself, prove the absence of every other policy or account-level restriction.

4.2 Role session: object access changed after assumption

The original notes record a denied GetObject request as devuser and a successful download after switching to BucketsAccessRole. The source download capture shows Image2.jpg and the browser completion notification; its unredacted header identifies the role session. The published crop retains the completed-download evidence.

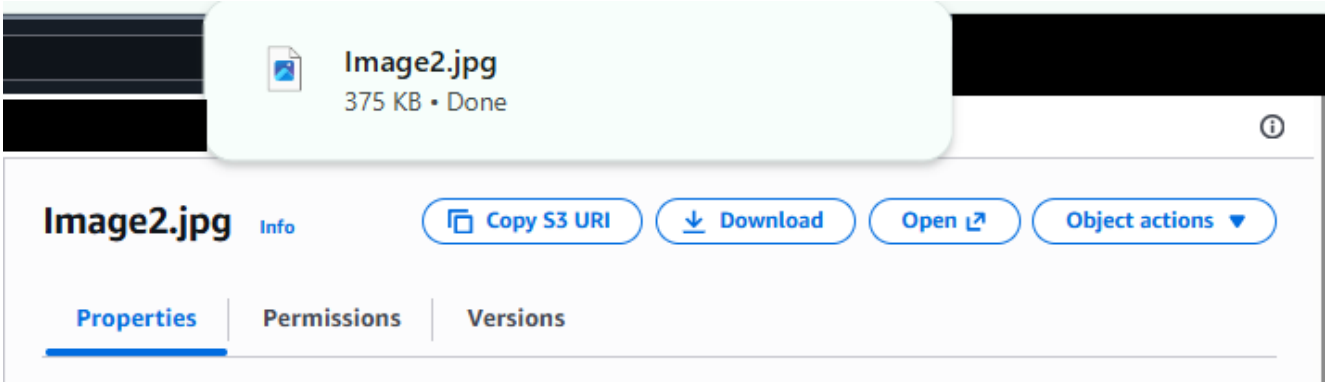


Figure 3. Image2.jpg appears in the browser download notification as Done. Account and bucket context is removed from this crop.

The earlier bucket2 policy capture names BucketsAccessRole as principal. Its first statement allows GetObject and PutObject on an object resource; its second allows ListBucket on the bucket. The account and complete resource names have been covered in the publication copy.



Figure 4. Earlier bucket2 resource-policy capture. Actions and role name remain visible; account numbers and resource identifiers are permanently covered in this PNG copy.

4.3 Bucket2 upload and the bucket3 policy

A later capture explicitly shows Upload succeeded while BucketsAccessRole is active, with Image2.jpg in the bucket object list. The earlier policy in Figure 4 explains the access pattern observed in that run. Because the later capture is from another training session, I do not present the two images as a complete policy snapshot of one uninterrupted session.

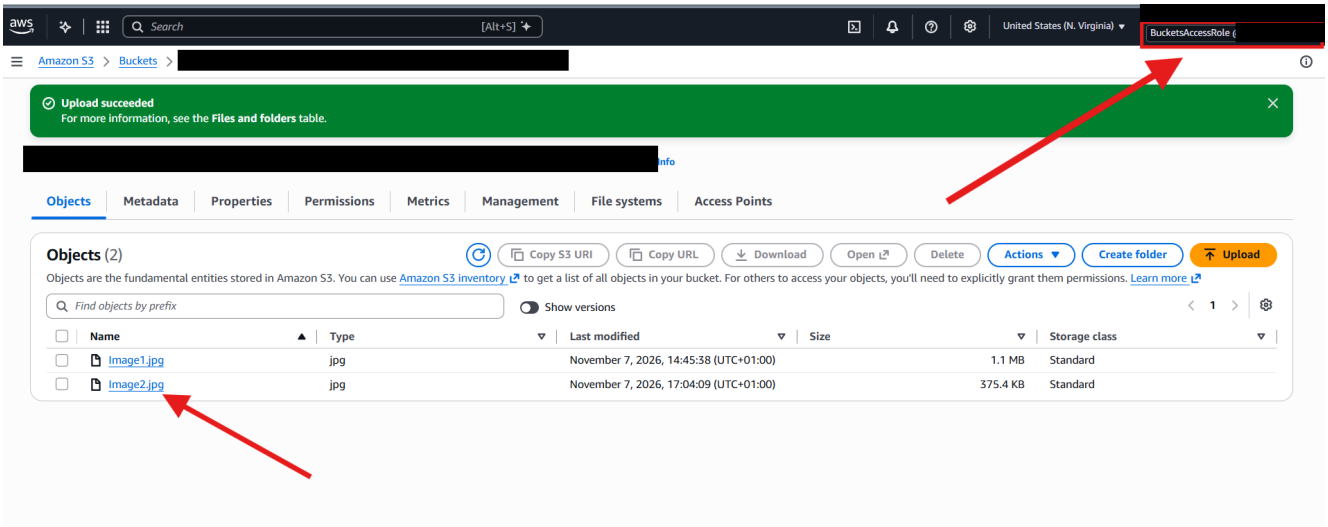


Figure 5. Later bucket2 upload capture: success banner, BucketsAccessRole session label, and Image2.jpg are visible. Account and generated bucket identifiers are covered.

The later bucket3 policy identifies a different principal: OtherBucketAccessRole. It contains object-read/object-write actions and a separate bucket-listing statement. The annotation was already present in the source image; the publication edit only hides identifiers.

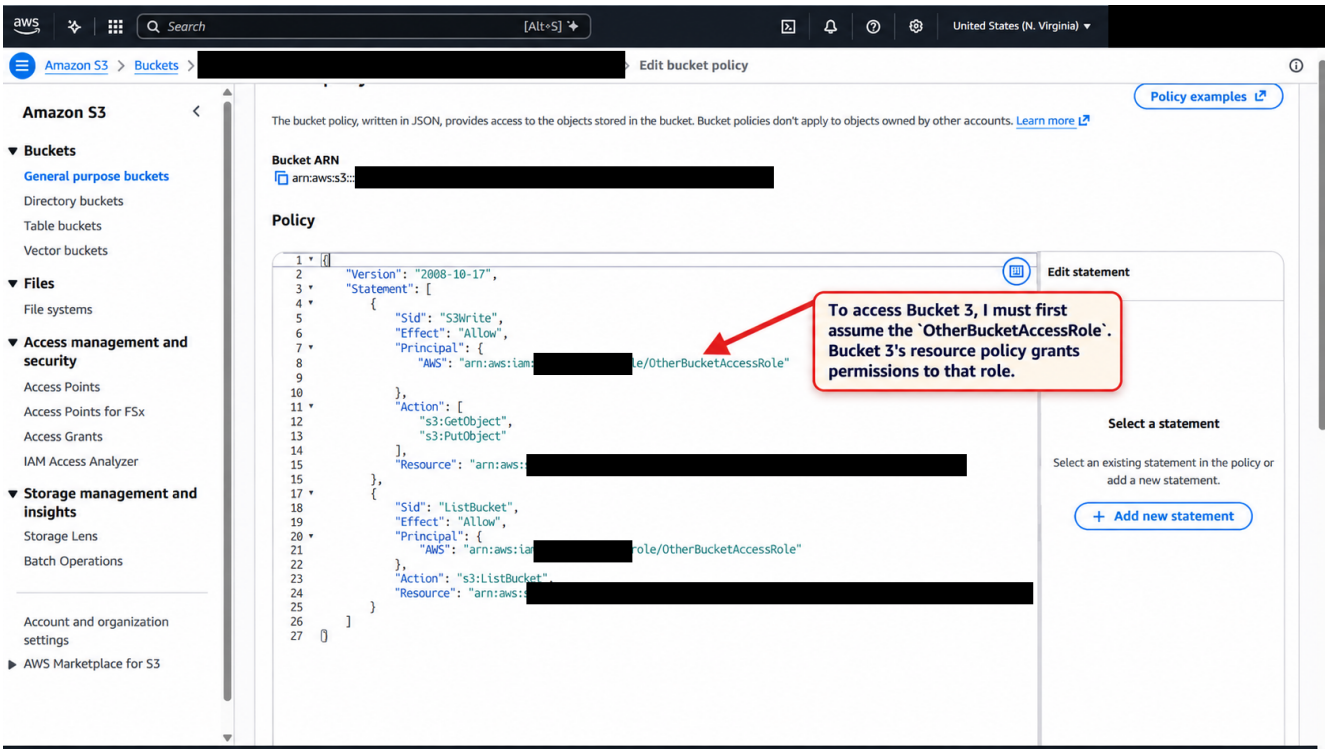


Figure 6. Bucket3 policy capture naming OtherBucketAccessRole. The supplied annotation points to that principal. Account and resource values are covered.

4.4 Bucket3 challenge: a successful upload is now evidenced

The older draft left the bucket3 outcome unverified. The newer capture shows OtherBucketAccessRole active, Upload succeeded, one file successfully uploaded, and Image2.jpg marked Succeeded. The unredacted destination matches the bucket3 policy capture from the same later training account.

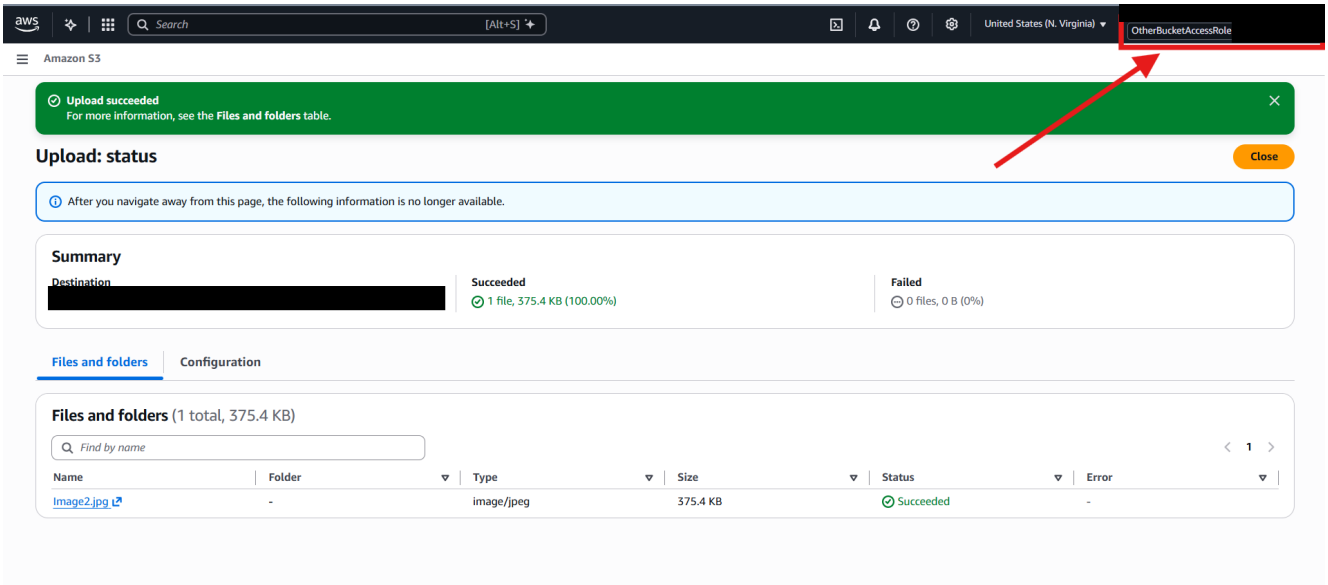


Figure 7. Later bucket3 upload summary: one file succeeded under OtherBucketAccessRole. The destination and account identifier are covered; the role name and result remain visible.

This supports the narrow claim that the captured Image2.jpg upload succeeded after the active role changed. It does not verify every possible object operation, the full trust/permission chain for that second role, or an unseen lab-grading result.

5. Technical explanation

S3 evaluates specific actions against appropriate resources. CreateBucket and PutObject are different permissions. ListObjects API requests use the ListBucket policy action; bucket operations use a bucket ARN, while object actions use an object ARN. The notes contain the label s3:ListObjects; I use the documented s3:ListBucket action here rather than publish that raw example as a deployable policy [1].

Within one account, an applicable resource-policy allow can authorize access that is absent from an identity policy. An applicable explicit deny overrides an allow; other policy limits can also matter. The reviewed bucket2 policy therefore provides a policy explanation for role-based object access without implying universal access to S3 [2].

A role trust policy controls who may assume the role, and STS returns temporary credentials. Requests made with those credentials use the role session context; switching roles is not an addition of all original user-group permissions to all role permissions [3, 4].

6. Limitations and lessons

The account identities, resource names and some Console context are deliberately removed from publication copies. Black rectangles cover pixels in flattened PNGs; they are not reversible overlays. Two figures are focused crops. No screenshot has been generated to fill an evidence gap, and no original vault image or note was changed.

The captured dates and account identifiers establish multiple training sessions. The report preserves that distinction instead of implying a continuous end-to-end trace. The successful bucket3 upload is newly supported by the later capture, but the full role-assumption policy for OtherBucketAccessRole and the lab grading outcome remain unverified.

I did not collect a CloudTrail export, a CLI transcript, or a complete inventory of permissions boundaries, session policies, Organizations controls, encryption-key policies or explicit denies. Written observations without an included figure are identified as such in Table 1. The raw course instructions and illustrative diagrams are not redistributed.

The practical review habit is to identify the current principal, requested action, resource and relevant identity/resource policies before explaining a result. For a stronger repeatable test, I would record the active principal and corresponding API outcomes in one controlled session. That is future work, not a completed test in this report.

References

[1] Amazon S3, Required permissions for Amazon S3 API operations.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-with-s3-policy-actions.html

[2] AWS IAM, Policy evaluation logic.
https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic.html

[3] AWS STS, AssumeRole.
https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRole.html

[4] AWS IAM, Permissions for AssumeRole, AssumeRoleWithSAML, and AssumeRoleWithWebIdentity.
https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_temp_control-access_assumerole.html

Evidence basis: private Obsidian observations and original Console captures reviewed for this publication. Training-instruction text is used only to identify the exercise context, not as proof that an operation was performed.